

Analysis: FreeCell Statistics

The first thing to notice in this problem is that both P_D and P_G can range from 0 to 100, so let's start by taking care of some easy cases. If our unnamed player has won 100% of his total games ($P_G = 100$) but not won 100% of his games today ($P_D < 100$), then something has clearly gone wrong with the calculator. Similarly, if $P_G = 0$ but $P_D > 0$, something is wonky.

The cases where $P_D = P_G = 0$ or $P_D = P_G = 100$ are also easy, and the answer for both of them is "Possible" — they mean that all the games ever ended with the same result (loss or win, respectively).

The trick now is that once we ruled out the case of P_G being zero or a hundred, we do not need to worry about it any more! Indeed, assume we have a solution that gives the correct daily percentage of wins, and it consists of W games won and L games lost today. To get a global win percentage of P_G , we can, for instance, assume we won a total of $(W + L) * P_G$ and lost a total of $(W + L) * (100 - P_G)$. As $1 \leq P_G \leq 99$, the numbers are greater than W and L , respectively, so they are possible to achieve.

The small data set can now be solved by brute force by simply trying all possible values of D from 1 to N and all possible number of games won from 0 to D , and seeing if any of them results in exactly P_D percentage of games being won.

Solving the large data set can require some maths. One way to solve the problem is to directly solve for the minimum number of games we would need to play in order to get a win percentage of P_D and simply verify that this number is $\leq N$. If we let W be the number of games we have won today, then we want to solve $W / D = P_D / 100$ for the minimum value of D such that W is integral.

From here, it is easy to see $D = 100 * W / P_D$. Thus, if we want to minimize D , then we need to find the smallest value W such that the right hand side is integral. In order to do this, we divide 100 and P_D by their [greatest common divisor](#) (let's call this value G) so that they are relatively prime and W is minimal when it is P_D / G . Plugging this in and cancelling terms tells us that $D = 100 / G$ is the fewest number of games we must be play.

A simpler way to solve this problem is by brute force. We can just try all possible values of D from 1 to N and check if any of them could result in exactly P_D percentage of games being won by checking that $D * P_D = 0 \pmod{100}$, stopping the loop the first time we find a candidate value of D or we exceed N games. While at first this solution appears to be $O(N)$ and would time out for the large data set, this loop will in fact run *at most* 100 times, regardless of the value of N , so this solution will be plenty fast enough. Coders who noticed this simpler solution early were rewarded with very fast submission times.

Analysis: Pseudominion

Let's denote the bonus numbers of a card c by $c.\text{card_draw}$, $c.\text{score}$ and $c.\text{turns}$. Let $c.\text{index}$ be the (zero based) index of the card in the ordering from the input file.

After looking at the problem constraints, we can classify a card c as:

- a T card, if $c.\text{turns} > 0$.
- a C_0 card, if $c.\text{turns} = 0$ and $c.\text{card_draw} = 0$
- a C_1 card, if $c.\text{turns} = 0$ and $c.\text{card_draw} = 1$
- a C_2 card, if $c.\text{turns} = 0$ and $c.\text{card_draw} = 2$

Let $T[i]$ denote the i -th T card in the sequence of all T cards sorted by the index. We define $C_0[i]$, $C_1[i]$ and $C_2[i]$ analogously.

The first key observation is that it never hurts to play a T card whenever we have one in our hand. We never lose turns or score and we may draw new cards by doing so.

Let's think about some arbitrary sequence of played cards. There are two observations we need to make before we can proceed.

1. Because C_0 cards do not add turns or cards to our hand, we can postpone playing them until we have no other cards we want to play. So we can transform any valid sequence of played cards into another sequence that has all C_0 cards at the very end. The total score, being the sum of individual scores of all played cards, will obviously remain the same.
2. Let's say there are two C_1 cards a and b such that we played a before b , but $b.\text{index} < a.\text{index}$. Because $b.\text{index} < a.\text{index}$, we already had both cards in our hand when we played card a . So we can instead play card b first and card a second. The resulting sequence with a and b swapped will remain valid because a and b both have the same card bonus (1 card) and the same turn bonus (0 turns). Furthermore, the score will remain the same. We can do the same for two C_2 cards.

The first observation shows us that we don't care about C_0 cards until the very end when we can use our remaining turns on whatever C_0 cards we have already drawn. Obviously, we should sort the C_0 cards in our hand by decreasing score bonus and then play as many as we can.

The second observation shows us that we can transform any optimal sequence of played cards into another one that has cards of the same type ordered by index. This means that we can play C_1 (and C_2) cards in increasing order by index.

So, whenever we have more than one C_1 (or C_2) cards in our hand we can always look at the one with the smallest index and choose whether we will play it right now or never at all.

These observations lead us to construct a directed acyclic weighted graph. A node represents a state of the game and is defined by these properties:

- hand - the number of drawn cards.
- turns - the number of turns left.
- t - $T[t]$ is the first T card that we haven't played yet.
- c_1 - $C_1[c_1]$ is the first C_1 card which we are not yet sure if we are going to play.

- $c_2 - C_2[c_2]$ is the first C_2 card which we are not yet sure if we are going to play.

An edge represents a valid transition from one game state to another and is usually associated with playing a card or deciding not to play a certain card at all. The weight of an edge indicates how much your score goes up when you switch between the two game states. For each node (hand, turns, t, c_1 , c_2), we add edges according to these rules:

1. If we have a T card in our hand, we can play it.
 - Condition: $T[t].index < hand$
 - Weight: $T[t].score$
 - Target node: $(\min(N, hand + T[t].card_draw), \min(N, turns + T[t].turns - 1), t + 1, c_1, c_2)$
2. If we have a C_1 card in our hand, we can play the first one.
 - Condition: $C_1[c_1].index < hand$
 - Weight: $C_1[c_1].score$
 - Target node: $(\min(N, hand + 1), turns - 1, t, c_1 + 1, c_2)$
3. If we have a C_1 card in our hand, we can throw away the first one.
 - Condition: $C_1[c_1].index < hand$
 - Weight: 0
 - Target node: $(hand, turns, t, c_1 + 1, c_2)$
4. If we have a C_2 card in our hand, we can play the first one.
 - Condition: $C_2[c_2].index < hand$
 - Weight: $C_2[c_2].score$
 - Target node: $(\min(N, hand + 2), turns - 1, t, c_1, c_2 + 1)$
5. If we have a C_2 card in our hand, we can throw away the first one.
 - Condition: $C_2[c_2].index < hand$
 - Weight: 0
 - Target node: $(hand, turns, t, c_1, c_2 + 1)$
6. We can always decide to finish the game by spending the remaining turns on C_0 cards.
This edge leads to the special final node and the weight of the edge is defined by the greedy algorithm that spends all remaining turns picking the best C_0 cards.

The answer to our problem is the length of the longest path in the graph described above. The graph is acyclic, so we can use dynamic programming or recursion with memoization to find the length of the longest path.

The number of nodes in the graph is $O(n^5)$, so the asymptotic time complexity of the algorithm is $O(n^5)$ too. In practice, the runtime of the program is really small because the vast majority of these states are unreachable. You can speed things up even more if you use the preceding observations to their full power. For example, if there is an edge corresponding to a T card, you should always follow it first.

Remark: This problem is inspired by the game Dominion, published by Rio Grande Games.

Analysis: The Killer Word

Every year, contestants come to us saying they implemented a program correctly but it ran too slowly. Unfortunately, being too slow is the same thing as being wrong. The Google Code Jam is an *algorithms* contest, and once you get past the early questions, coming up with a fast algorithm is often the whole point of the problem.

Why do we bring this up? Because this problem is a trap. On first glance, it looks just like the first two problems from the Qualification Round. We tell you an algorithm, and you implement it exactly as described:

- Loop through every one of Sean's lists.
- Loop through every word in the dictionary.
- See how many mistakes Sean makes while guessing this word.

Since the last step requires iterating over every word again, the running time here is $O(N^2M)$.

We will always give you the hardest inputs we can fit within the stated limits, which means you *will* see $N = 10000$ and $M = 100$. Neither a fast computer nor a fast language will save you here - the whole approach is simply too slow. For example, our C++ implementation takes over 20 minutes for a single test case. To succeed on an algorithms contest, you need to see these issues in advance, either by looking at the Big O complexity or by trying a worst-case input of your own.

Once you do see the issue, it isn't too hard to dramatically speed things up. The main idea is to combine the last two steps:

- Loop through every one of Sean's lists.
- Divide the words into different classes by length. If you focus on a single class, Sean will make the same first guess for any word in that class. This is because he has exactly the same information in each case.
- For each class, figure out what letter Sean will guess, and further divide the class based on each of the different responses you could give to Sean's guess.
- Repeat for each of the sub-classes until you are down to a single word, at which point Sean will finish with no mistakes.

The running time here is $O(NM)$, and it will go hundreds of times faster than the more straightforward algorithm.

This technique is similar to Dynamic Programming - the game begins in the same way for many different words, so we should not have to redo all that work every time.

Fun facts: this problem was originally called Bloodthirsty Executioner Man, but was renamed because the hangman is made out of paper, and has no blood. Sean himself answered several of the clarifications for this problem.